

实验三：存储管理

陈天乐 无 25

2025 年 6 月 3 日

目录

1	实验题目：动态分区存储管理	2
1.1	问题描述	2
1.2	实现要求	2
1.3	程序设计思路	2
1.3.1	内存分区节点结构体定义	2
1.3.2	MemoryManageUnit(MMU) 类	3
1.3.3	菜单函数	6
1.3.4	主函数	7
2	实验结果	7
3	思考题	9
4	实验总结	10
5	源代码	11

1 实验题目：动态分区存储管理

本实验选择了动态分区存储管理，操作系统平台选为 Windows，实现语言为 C++。

1.1 问题描述

分区存储管理有固定分区和动态分区两种方法。固定分区把内存划分为若干个固定大小的连续分区，每个分区的边界固定；而动态分区并不预先将内存事先划分成分区，当程序需要装入内存时系统从空闲的内存区中，采用不同的分配算法分配大小等于程序所需的内存空间。它有效地克服了固定分区方式中，由于分区内部剩余内存空置造成浪费的问题。

请基于空闲内存分区链表的存储管理，设计一个动态分区存储管理程序，支持包括首次适配法、下次适配法、最佳适配法和最坏适配法在内的不同分区分配算法。

1.2 实现要求

1. 维护一个记录已分配内存分区和空闲内存分区的链表；
2. 设计申请、释放函数循环处理用户的请求；
3. 实现首次适配法、下次适配法、最佳适配法和最坏适配法四种分区分配算法；
4. 可视化展示内存使用情况。

1.3 程序设计思路

首先对内存分区定义一个结构体，因为是用链表管理，因此包含起始地址、分区大小、分区状态（占用还是空闲）、指向下一块分区的指针。然后定义一个 MMU 类，用于进行内存分区的管理。该类中的函数可以实现在分区后插入、删除新分区，并可以归并空闲分区。同时按照题目要求，实现首次适配法、下次适配法、最佳适配法和最坏适配法在内的四种分区分配算法。最后编写一个可视化函数，用于和用户交互。用户可以自行定义内存大小、对内存分区的分配（还可以选择使用什么算法分配）和释放、显示当前内存分配情况等操作。

1.3.1 内存分区节点结构体定义

```
1 enum Status {FREE, ALLOCATED};
2
3 // 内存分区节点结构体
4 struct Partition {
5     int start;        // 起始地址
6     int size;         // 分区大小
7     Status status;    // 分区状态
8     Partition* next;  // 下一个分区指针
9
10    Partition(int s, int size, Status status)
11        : start(s), size(size), status(status), next(nullptr) {
12    }
13};
```

这一部分代码定义了内存分区结构体，包含了每个内存分区的基本信息。

1.3.2 MemoryManageUnit(MMU) 类

```
1 class MemoryManageUnit {
2 private:
3     int total_size;           //内存总大小
4     Partition* head;         //表头
5     Partition* last_used;     //下次适配算法的起始位置
6
7     void insertbehind(Partition* prev, Partition* newNode) {
8         newNode->next = prev->next;
9         prev->next = newNode;
10    }
11
12    void deletenode(Partition* prev, Partition* target) {
13        prev->next = target->next;
14        delete target;
15    }
16
17    // 合并相邻空闲分区
18    void mergeFree(Partition* node) {
19        //合并后驱空间分区
20        while (node->next && node->next->status == FREE) {
21            node->size += node->next->size;
22            Partition* temp = node->next;
23            node->next = temp->next;
24            delete temp;
25        }
26        //合并前驱空间分区
27        Partition* current = head;
28        Partition* prev = nullptr;
29        while (current) {
30            if (current == node && prev && prev->status == FREE) {
31                prev->size += node->size;
32                prev->next = node->next;
33                delete node;
34                node = prev; // 更新为合并后的前驱节点
35            }
36            prev = current;
37            current = current->next;
38        }
39    }
40
41    // 首次适配算法，找到合适的空闲区就分配
42    Partition* firstFit(int size) {
43        Partition* current = head;
44        while (current) {
45            if (current->status == FREE && current->size >= size) {
46                return current;
47            }
48            current = current->next;
49        }
50        return nullptr;
51    }
52
53    // 下次适配算法，从上次分配的位置开始查找下一个满足要求的空闲区
54    Partition* nextFit(int size) {
55        Partition* current = last_used->next ? last_used->next : head;
56        while (true) {
57            if (current->status == FREE && current->size >= size) {
58                last_used = current; // 更新上次结束位置
59                return current;
60            }
61        }
62    }
63 }
```

```
61         if (current == last_used) break; // 遍历完所有节点
62         current = current->next;
63     }
64     return nullptr;
65 }
66
67 // 最佳适配算法，找到大小最合适（最小的）的空闲区
68 Partition* bestFit(int size) {
69     Partition* best = nullptr;
70     int minSize = INT_MAX;
71     Partition* current = head;
72     while (current) {
73         if (current->status == FREE && current->size >= size) {
74             if (current->size < minSize) {
75                 minSize = current->size;
76                 best = current;
77             }
78         }
79         current = current->next;
80     }
81     return best;
82 }
83
84 // 最坏适配算法，找到最大的空闲区分配
85 Partition* worstFit(int size) {
86     Partition* worst = nullptr;
87     int maxSize = -1;
88     Partition* current = head;
89     while (current) {
90         if (current->status == FREE && current->size >= size) {
91             if (current->size > maxSize) {
92                 maxSize = current->size;
93                 worst = current;
94             }
95         }
96         current = current->next;
97     }
98     return worst;
99 }
100
101 public:
102     // 构造函数
103     MemoryManageUnit(int total_size = 1024) : total_size(total_size) {
104         head = new Partition(0, total_size, FREE);
105         last_used = head;
106     }
107
108     // 析构函数
109     ~MemoryManageUnit() {
110         Partition* current = head;
111         while (current) {
112             Partition* temp = current;
113             current = current->next;
114             delete temp;
115         }
116     }
117
118     // 内存分配接口
119     bool allocate(int size, const string& algorithm) {
120         if (size <= 0) {
121             cout << "错误：申请内存大小必须大于0!" << endl;
122             return false;
123         }
```

```

123     }
124
125     Partition* found = nullptr; //找到合适的空闲分区
126     if (algorithm == "first_fit") found = firstFit(size);
127     else if (algorithm == "next_fit") found = nextFit(size);
128     else if (algorithm == "best_fit") found = bestFit(size);
129     else if (algorithm == "worst_fit") found = worstFit(size);
130     else {
131         cout << "错误：未知的分配算法!" << endl;
132         return false;
133     }
134
135     if (!found) {
136         cout << "错误：无足够空间分配!" << size << endl;
137         return false;
138     }
139
140     // 分割空闲分区
141     if (found->size > size) {
142         Partition* newPartition = new Partition(found->start + size, found->size - size,
143         FREE);
144         insertbehind(found, newPartition);
145         found->size = size;
146     }
147
148     found->status = ALLOCATED;
149     return true;
150 }
151
152 // 内存释放接口
153 bool free(int start) {
154     Partition* current = head;
155     Partition* prev = nullptr;
156     while (current) {
157         if (current->status == ALLOCATED && current->start == start) {
158             current->status = FREE;
159             mergeFree(current); // 合并相邻空闲分区
160             return true;
161         }
162         prev = current;
163         current = current->next;
164     }
165     cout << "错误：未找到起始地址为 " << start << " 的已分配分区!" << endl;
166     return false;
167 }
168
169 // 可视化内存状态函数
170 void visualize() {
171     cout << "\n内存使用情况（地址从低到高）：" << endl;
172     cout << string(50, '=') << endl;
173     Partition* current = head;
174     while (current) {
175         string status = (current->status == ALLOCATED) ? "已分配" : "空闲";
176         cout << "地址：" << current->start << "-"
177             << current->start + current->size - 1 << " | "
178             << "大小：" << current->size << " KB" << " | 状态：" << status << endl;
179         current = current->next;
180     }
181     cout << string(50, '=') << "\n" << endl;
182 }
183 };

```



```

53         }
54         case 3: // 显示状态
55             mmu.visualize();
56             cin.clear();
57             break;
58         case 4: // 退出程序
59             cout << "感谢使用，程序已退出！" << endl;
60             cin.clear();
61             return;
62     }
63 }
64 }
65 else {
66     // 清除输入错误状态
67     cin.clear();
68     // 忽略输入缓冲区中的剩余字符
69     cin.ignore(INT_MAX, '\n');
70     cout << "输入无效，请重新输入1-4的数字！" << endl;
71 }
72 }
73 }

```

这一函数是菜单函数，用于可视化内存分配情况，以及和用户实现交互。

1.3.4 主函数

```

1 int main() {
2
3     int init_mem_size;
4     cout << "初始化内存大小（默认单位为KB）：";
5     cin >> init_mem_size;
6     MemoryManageUnit mmu(init_mem_size); // 默认以KB为内存单位
7     menu(mmu);
8     return 0;
9 }

```

在主函数中，可以定义内存初始大小，并调用菜单函数实现用户和系统交互。

2 实验结果

实验结果已经保存在 result.txt 中，助教可以自行运行程序使用，示例结果如下：

```

1 初始化内存大小（默认单位为KB）： 2048
2
3 ===== 内存管理菜单 =====
4 1. 分配内存 (Allocate memory)
5 2. 释放内存 (Deallocate memory)
6 3. 显示内存分配情况 (Display memory allocation status)
7 4. 退出 (Exit)
8 请输入选项（1-4）： 1
9
10 请输入要分配的内存大小（默认单位为KB）： 700
11 请选择分配算法（first_fit/next_fit/best_fit/worst_fit）： best_fit
12 已分配成功！当前的内存分配情况为：
13
14 内存使用情况（地址从低到高）：
15 =====
16 地址：0-699 | 大小：700 KB | 状态：已分配
17 地址：700-2047 | 大小：1348 KB | 状态：空闲
18 =====

```

```
19
20
21 ===== 内存管理菜单 =====
22 1. 分配内存 (Allocate memory)
23 2. 释放内存 (Deallocate memory)
24 3. 显示内存分配情况 (Display memory allocation status)
25 4. 退出 (Exit)
26 请输入选项 (1-4) : 1
27
28 请输入要分配的内存大小 (默认单位为KB) : 800
29 请选择分配算法 (first_fit/next_fit/best_fit/worst_fit) : best_fit
30 已分配成功! 当前的内存分配情况为:
31
32 内存使用情况 (地址从低到高):
33 =====
34 地址: 0-699 | 大小: 700 KB | 状态: 已分配
35 地址: 700-1499 | 大小: 800 KB | 状态: 已分配
36 地址: 1500-2047 | 大小: 548 KB | 状态: 空闲
37 =====
38
39
40 ===== 内存管理菜单 =====
41 1. 分配内存 (Allocate memory)
42 2. 释放内存 (Deallocate memory)
43 3. 显示内存分配情况 (Display memory allocation status)
44 4. 退出 (Exit)
45 请输入选项 (1-4) : 2
46
47 请输入要释放分区的起始地址: 0
48 已释放成功! 当前的内存分配情况为:
49
50 内存使用情况 (地址从低到高):
51 =====
52 地址: 0-699 | 大小: 700 KB | 状态: 空闲
53 地址: 700-1499 | 大小: 800 KB | 状态: 已分配
54 地址: 1500-2047 | 大小: 548 KB | 状态: 空闲
55 =====
56
57
58 ===== 内存管理菜单 =====
59 1. 分配内存 (Allocate memory)
60 2. 释放内存 (Deallocate memory)
61 3. 显示内存分配情况 (Display memory allocation status)
62 4. 退出 (Exit)
63 请输入选项 (1-4) : 1
64
65 请输入要分配的内存大小 (默认单位为KB) : 500
66 请选择分配算法 (first_fit/next_fit/best_fit/worst_fit) : best_fit
67 已分配成功! 当前的内存分配情况为:
68
69 内存使用情况 (地址从低到高):
70 =====
71 地址: 0-699 | 大小: 700 KB | 状态: 空闲
72 地址: 700-1499 | 大小: 800 KB | 状态: 已分配
73 地址: 1500-1999 | 大小: 500 KB | 状态: 已分配
74 地址: 2000-2047 | 大小: 48 KB | 状态: 空闲
75 =====
76
77
78 ===== 内存管理菜单 =====
79 1. 分配内存 (Allocate memory)
80 2. 释放内存 (Deallocate memory)
```

```

81 3. 显示内存分配情况(Display memory allocation status)
82 4. 退出(Exit)
83 请输入选项 (1-4) : 1
84
85 请输入要分配的内存大小 (默认单位为KB) : 30
86 请选择分配算法 (first_fit/next_fit/best_fit/worst_fit) : worst_fit
87 已分配成功! 当前的内存分配情况为:
88
89 内存使用情况 (地址从低到高):
90 =====
91 地址: 0-29 | 大小: 30 KB | 状态: 已分配
92 地址: 30-699 | 大小: 670 KB | 状态: 空闲
93 地址: 700-1499 | 大小: 800 KB | 状态: 已分配
94 地址: 1500-1999 | 大小: 500 KB | 状态: 已分配
95 地址: 2000-2047 | 大小: 48 KB | 状态: 空闲
96 =====
97
98
99 ===== 内存管理菜单 =====
100 1. 分配内存 (Allocate memory)
101 2. 释放内存 (Deallocate memory)
102 3. 显示内存分配情况 (Display memory allocation status)
103 4. 退出 (Exit)
104 请输入选项 (1-4) : 1
105
106 请输入要分配的内存大小 (默认单位为KB) : 100
107 请选择分配算法 (first_fit/next_fit/best_fit/worst_fit) : next_fit
108 已分配成功! 当前的内存分配情况为:
109
110 内存使用情况 (地址从低到高):
111 =====
112 地址: 0-29 | 大小: 30 KB | 状态: 已分配
113 地址: 30-129 | 大小: 100 KB | 状态: 已分配
114 地址: 130-699 | 大小: 570 KB | 状态: 空闲
115 地址: 700-1499 | 大小: 800 KB | 状态: 已分配
116 地址: 1500-1999 | 大小: 500 KB | 状态: 已分配
117 地址: 2000-2047 | 大小: 48 KB | 状态: 空闲
118 =====
119
120
121 ===== 内存管理菜单 =====
122 1. 分配内存 (Allocate memory)
123 2. 释放内存 (Deallocate memory)
124 3. 显示内存分配情况 (Display memory allocation status)
125 4. 退出 (Exit)
126 请输入选项 (1-4) : 4
127 感谢使用, 程序已退出!

```

可以从结果看出, 该程序的各个功能以及四种分配算法是正确的。

3 思考题

1. 基于位图和空闲链表的存储管理各有什么优劣? 如果使用基于位图的存储管理, 有何额外注意事项?

使用位图管理的优势是空间利用效率高, 仅用 1bit 表示占用状态, 内存占用极小; 可以通过位运算直接定位某块是否空闲, 时间复杂度为 $O(1)$; 对于固定块大小的系统, 管理逻辑非常简单。但是缺点同样很明显, 分配大块空间时效率极低, 需要扫描位图寻找连续的“0”位; 并且无法直接支持可变大小的存储块。

使用空闲链表管理的优势是支持可变大小块，适合动态内存分配；并且分配算法灵活，可结合不同算法优化分配效率，减少内存碎片；在释放块时可快速合并相邻空闲块，减少外碎片。缺点是链表查找的效率较低，性能随链表长度增加而下降明显；同时相较于位图管理，每个节点需存储地址、大小、指针等信息，链表长度随空闲块数增加而膨胀；对于维护链表的代价也更高，逻辑也更复杂。

若使用位图管理，需要注意切割内存片的大小。块大小过小将导致位图过大，过大则浪费空间（产生大量内碎片）。同时要注意，块地址需与块大小对齐。在多线程或多进程环境下，位图的修改必须加互斥锁保护，防止多个操作同时修改同一位造成错误。

4 实验总结

本次实验中，我通过实现动态内存分区管理对内存的分区管理细节有了更进一步的了解。在理解了理论的同时，使用 C++ 具体实现，并且完成了首次适配法、下次适配法、最佳适配法和最坏适配法四种算法的实现，在测试中也感受到了不同分配算法对内存分配的结果有何影响。在完成试验后，还对固定分区、空闲链表管理的性能和效果做了一定的分析。在最后做思考题时也对课上所学的位图、空闲链表管理的知识做了回顾。未来可以在此基础上实现伙伴系统，对比不同算法在内存碎片与分配速度上的差异。

最后，感谢老师和助教的批阅！

5 源代码

```

1  #include <iostream>
2  #include <string>
3  #include <climits>
4
5  using namespace std;
6
7  enum Status {FREE, ALLOCATED};
8
9  // 内存分区节点结构体
10 struct Partition {
11     int start;        // 起始地址
12     int size;         // 分区大小
13     Status status;    // 分区状态
14     Partition* next;  // 下一个分区指针
15
16     Partition(int s, int size, Status status)
17         : start(s), size(size), status(status), next(nullptr) {}
18 };
19
20
21
22 class MemoryManageUnit {
23 private:
24     int total_size;        // 内存总大小
25     Partition* head;       // 表头
26     Partition* last_used;  // 下次适配算法的起始位置
27
28     void insertbehind(Partition* prev, Partition* newNode) {
29         newNode->next = prev->next;
30         prev->next = newNode;
31     }
32
33     void deletenode(Partition* prev, Partition* target) {
34         prev->next = target->next;
35         delete target;
36     }
37
38     // 合并相邻空闲分区
39     void mergeFree(Partition* node) {
40         // 合并后驱空间分区
41         while (node->next && node->next->status == FREE) {
42             node->size += node->next->size;
43             Partition* temp = node->next;
44             node->next = temp->next;
45             delete temp;
46         }
47         // 合并前驱空间分区
48         Partition* current = head;
49         Partition* prev = nullptr;
50         while (current) {
51             if (current == node && prev && prev->status == FREE) {
52                 prev->size += node->size;
53                 prev->next = node->next;
54                 delete node;
55                 node = prev; // 更新为合并后的前驱节点
56             }
57             prev = current;
58             current = current->next;
59         }

```

```
60     }
61
62     // 首次适配算法，找到合适的空闲区就分配
63     Partition* firstFit(int size) {
64         Partition* current = head;
65         while (current) {
66             if (current->status == FREE && current->size >= size) {
67                 return current;
68             }
69             current = current->next;
70         }
71         return nullptr;
72     }
73
74     // 下次适配算法，从上次分配的位置开始查找下一个满足要求的空闲区
75     Partition* nextFit(int size) {
76         Partition* current = last_used->next ? last_used->next : head;
77         while (true) {
78             if (current->status == FREE && current->size >= size) {
79                 last_used = current; // 更新上次结束位置
80                 return current;
81             }
82             if (current == last_used) break; // 遍历完所有节点
83             current = current->next;
84         }
85         return nullptr;
86     }
87
88     // 最佳适配算法，找到大小最合适（最小的）的空闲区
89     Partition* bestFit(int size) {
90         Partition* best = nullptr;
91         int minSize = INT_MAX;
92         Partition* current = head;
93         while (current) {
94             if (current->status == FREE && current->size >= size) {
95                 if (current->size < minSize) {
96                     minSize = current->size;
97                     best = current;
98                 }
99             }
100             current = current->next;
101         }
102         return best;
103     }
104
105     // 最坏适配算法，找到最大的空闲区分配
106     Partition* worstFit(int size) {
107         Partition* worst = nullptr;
108         int maxSize = -1;
109         Partition* current = head;
110         while (current) {
111             if (current->status == FREE && current->size >= size) {
112                 if (current->size > maxSize) {
113                     maxSize = current->size;
114                     worst = current;
115                 }
116             }
117             current = current->next;
118         }
119         return worst;
120     }
121 }
```

```

122 public:
123     //构造函数
124     MemoryManageUnit(int total_size = 1024) : total_size(total_size) {
125         head = new Partition(0, total_size, FREE);
126         last_used = head;
127     }
128
129     //析构函数
130     ~MemoryManageUnit() {
131         Partition* current = head;
132         while (current) {
133             Partition* temp = current;
134             current = current->next;
135             delete temp;
136         }
137     }
138
139     //内存分配接口
140     bool allocate(int size, const string& algorithm) {
141         if (size <= 0) {
142             cout << "错误: 申请内存大小必须大于0!" << endl;
143             return false;
144         }
145
146         Partition* found = nullptr; //找到合适的空闲分区
147         if (algorithm == "first_fit") found = firstFit(size);
148         else if (algorithm == "next_fit") found = nextFit(size);
149         else if (algorithm == "best_fit") found = bestFit(size);
150         else if (algorithm == "worst_fit") found = worstFit(size);
151         else {
152             cout << "错误: 未知的分配算法!" << endl;
153             return false;
154         }
155
156         if (!found) {
157             cout << "错误: 无足够空间分配! " << size << endl;
158             return false;
159         }
160
161         // 分割空闲分区
162         if (found->size > size) {
163             Partition* newPartition = new Partition(found->start + size, found->size - size,
164             FREE);
165             insertbehind(found, newPartition);
166             found->size = size;
167         }
168
169         found->status = ALLOCATED;
170         return true;
171     }
172
173     // 内存释放接口
174     bool free(int start) {
175         Partition* current = head;
176         Partition* prev = nullptr;
177         while (current) {
178             if (current->status == ALLOCATED && current->start == start) {
179                 current->status = FREE;
180                 mergeFree(current); // 合并相邻空闲分区
181                 return true;
182             }
183             prev = current;

```



```
245         cin >> start;
246         bool flag = mmu.free(start);
247         if (flag)
248         {
249             cout << "已释放成功！当前的内存分配情况为：" << endl;
250             mmu.visualize();
251             cin.clear();
252         }
253         else
254         {
255             cout << "出现异常错误，释放失败！" << endl;
256             cin.clear();
257         }
258         break;
259     }
260     case 3: // 显示状态
261         mmu.visualize();
262         cin.clear();
263         break;
264     case 4: // 退出程序
265         cout << "感谢使用，程序已退出！" << endl;
266         cin.clear();
267         return;
268     }
269 }
270 }
271 else {
272     // 清除输入错误状态
273     cin.clear();
274     // 忽略输入缓冲区中的剩余字符
275     cin.ignore(INT_MAX, '\n');
276     cout << "输入无效，请重新输入1-4的数字！" << endl;
277 }
278 }
279 }
280
281 int main() {
282
283     int init_mem_size;
284     cout << "初始化内存大小（默认单位为KB）：" << endl;
285     cin >> init_mem_size;
286     MemoryManageUnit mmu(init_mem_size); // 默认以KB为内存单位
287     menu(mmu);
288     return 0;
289 }
```

Listing 1: main.cpp